

CS3033 - Theory of Computation

The beginning

Angel Napa

1.



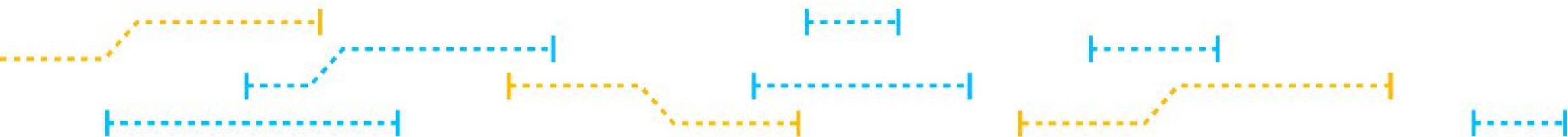
Presentación

Curso y Docente

The *professor*



- **Name:** Angel Napa
- **Degree:** Master in Mathematics:
Institute of Mathematics Pure and Applied(IMPA), Brasil
- **Area:** Combinatorics, Algorithms
- **Group:** ALCOMB
- **Subarea:**
 - Combinatorics Optimization
 - Analysis of Algorithms
 - Graph Theory
 - Operations Research



Evaluation *System*

Evaluación	Teoría	Laboratorio
	3 exámenes (E):	
	Examen E1 (25%)	Evaluación continua C1 (25%)
	Examen E2 (25%)	
	Examen E3 (25%)	
	75%	25%
	100%	



¿What is *Theory of Computation*?

- What are the **fundamental** capabilities and limitations of computers?
 - 1930: What is computation? - Mathematical Logicians



Three areas *interpretations*

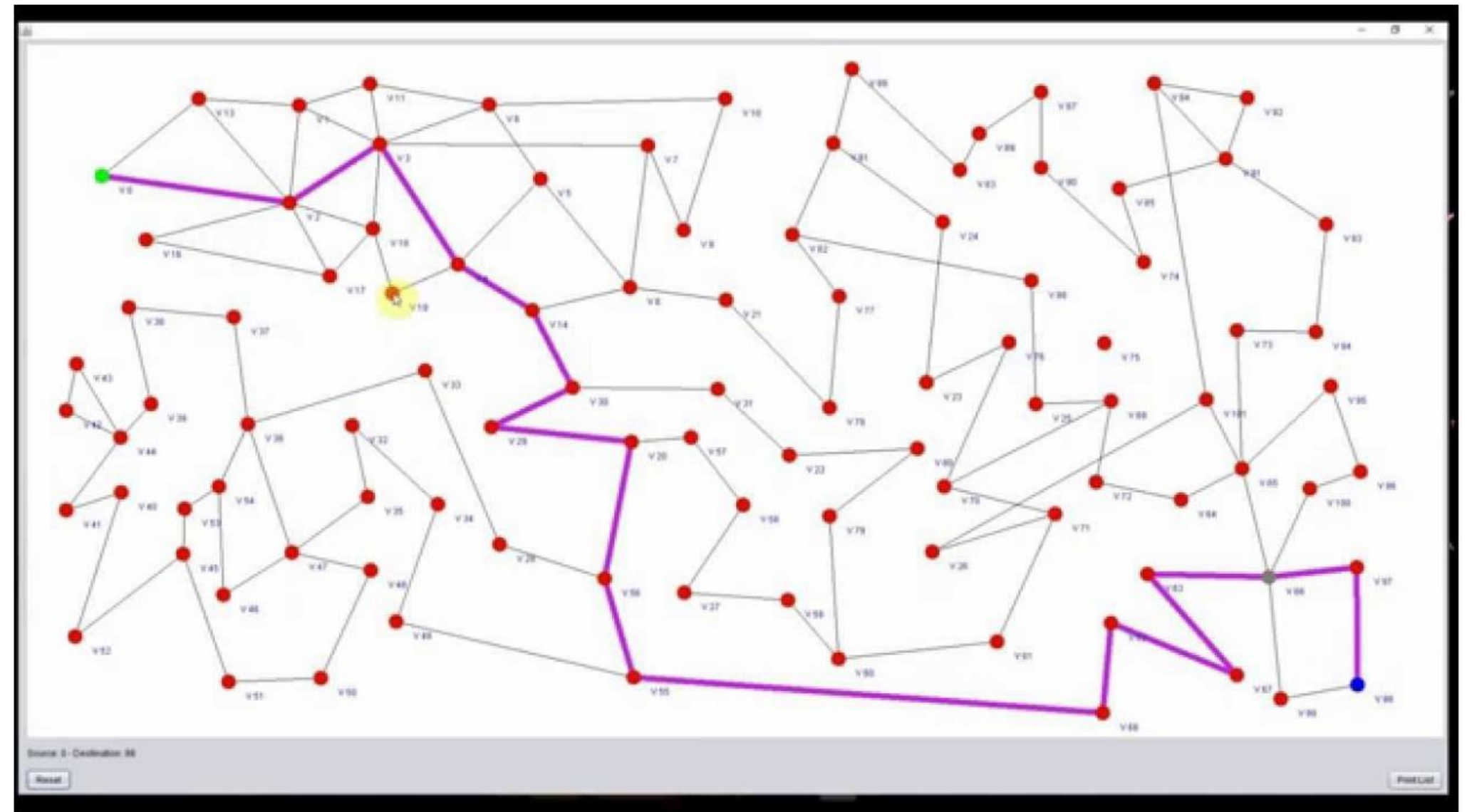
- **Complexity Theory**
 - What makes a problem computationally hard or easy?
- **Computability Theory**
 - What makes a problem computationally solvable?
- **Automata Theory**
 - First idea of an abstract simple machine

Complexity *Theory*

- **Hard vs easy**
 - Easy problem: sorting elements
 - Hard problem: Create a Valid Schedule for a University
 - What makes a problem computationally solvable?
- **WHAT** makes some problems easier and others hard?

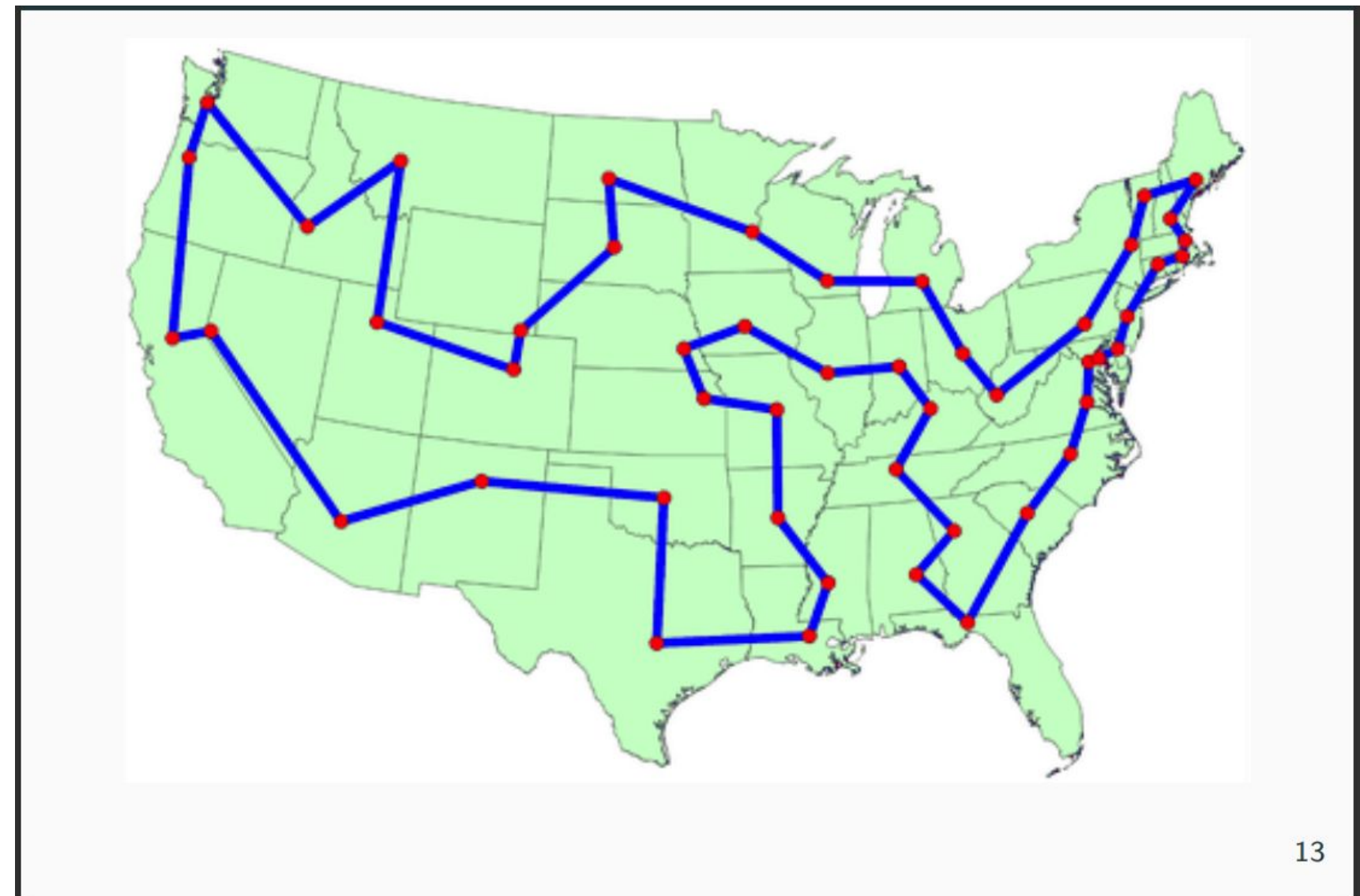
Complexity *Theory*

- Some **easy** problems
- (almost) **All AED problems**
 - Sorting
 - Shortest Path
 - Minimum Span Tree
 - etc.



Complexity *Theory*

- Some **hard** problems
 - TSP
 - Hamilton Cycle
 - Scheduling
 - 3SAT

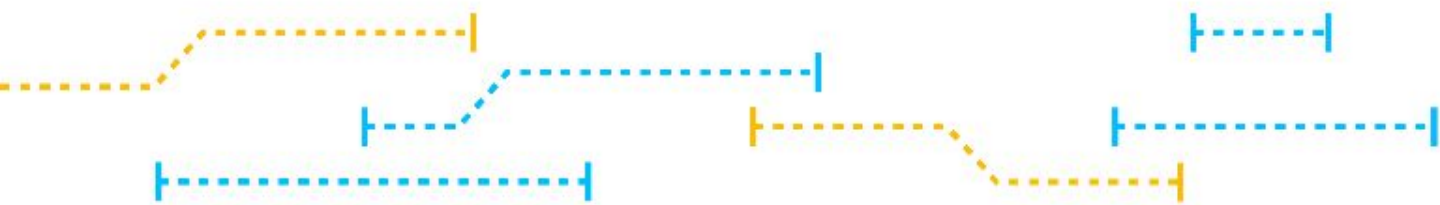
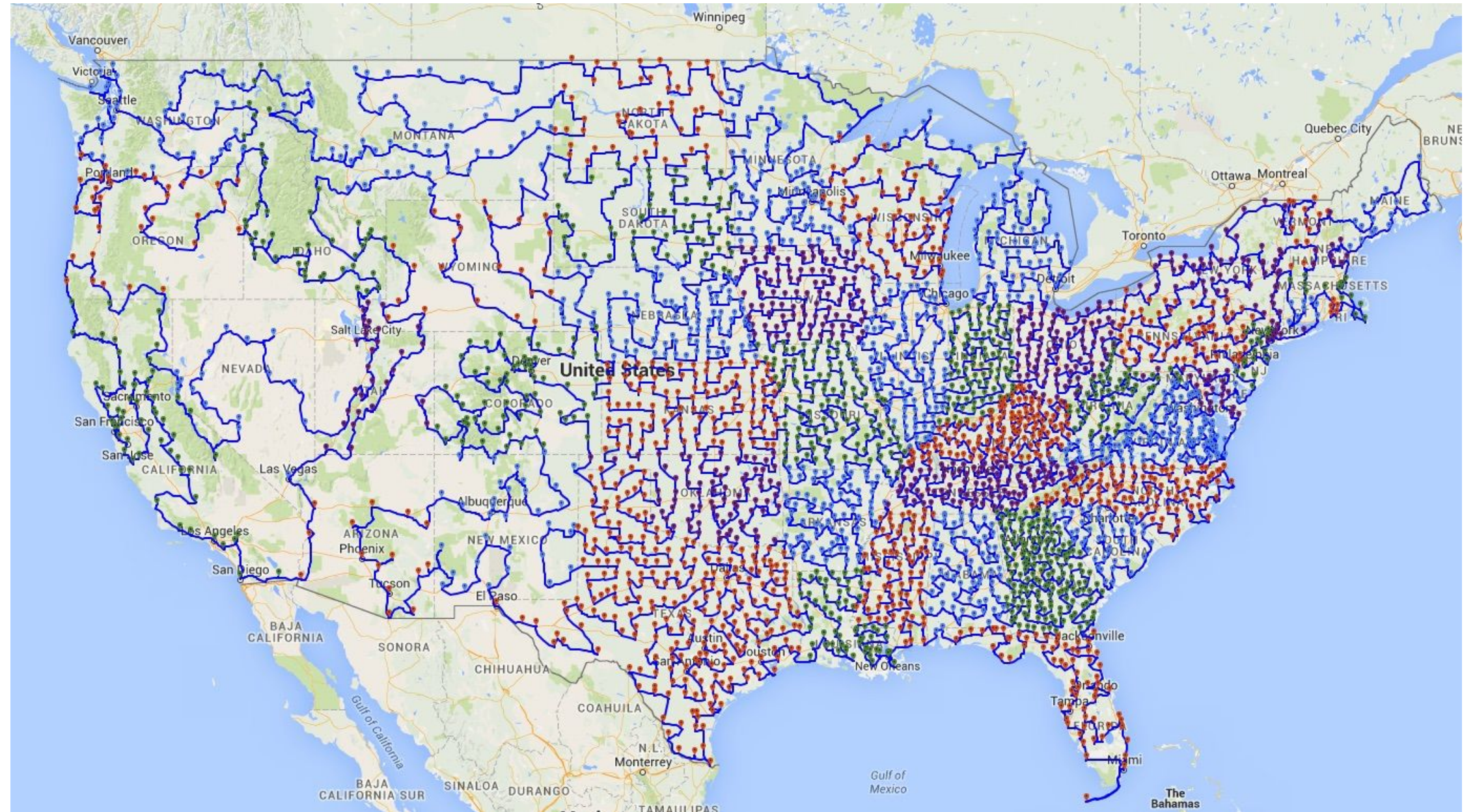


13

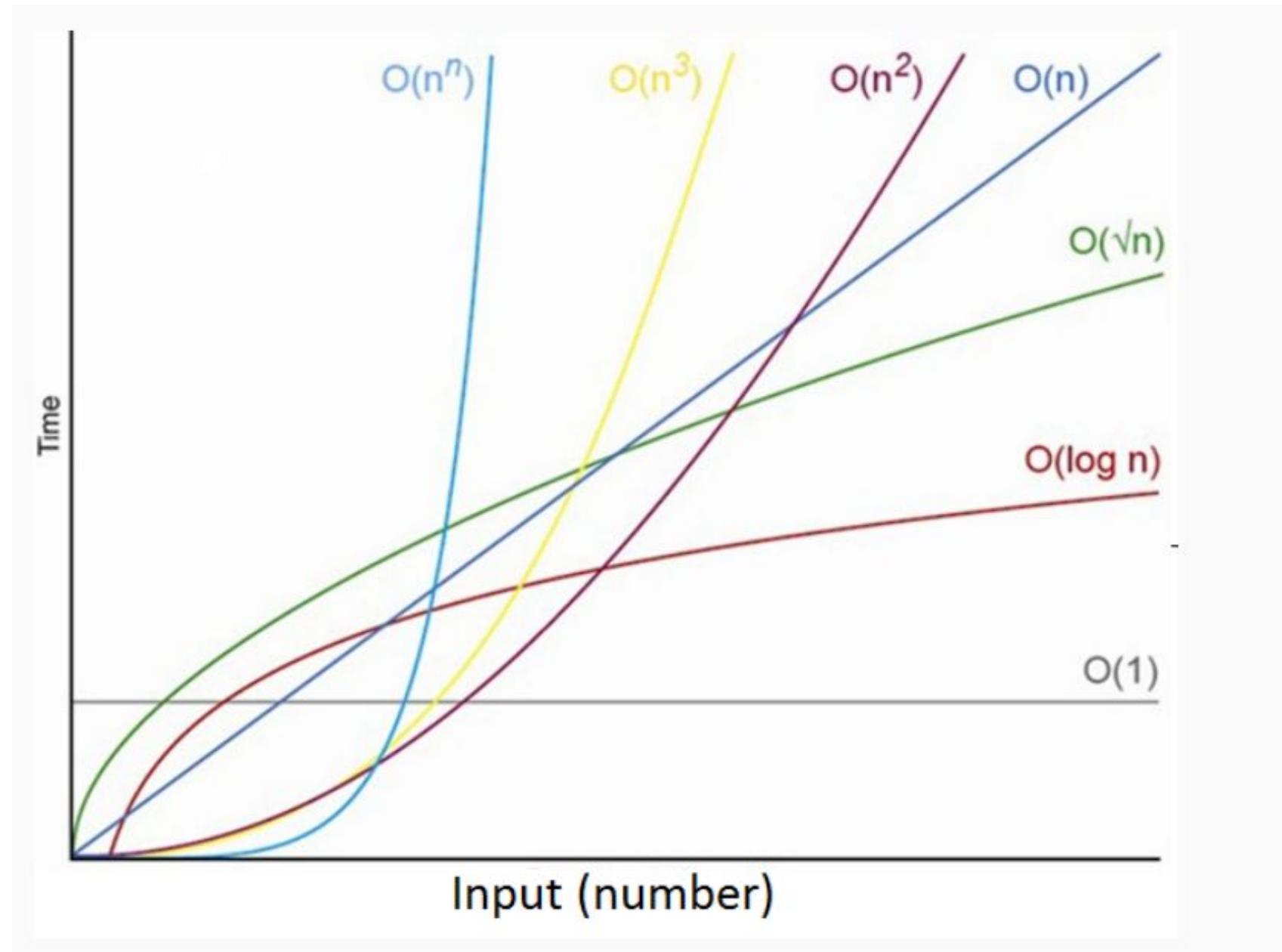


Complexity *Theory*

- Some **hard** problems
 - TSP
 - Hamilton Cycle
 - Scheduling
 - 3SAT



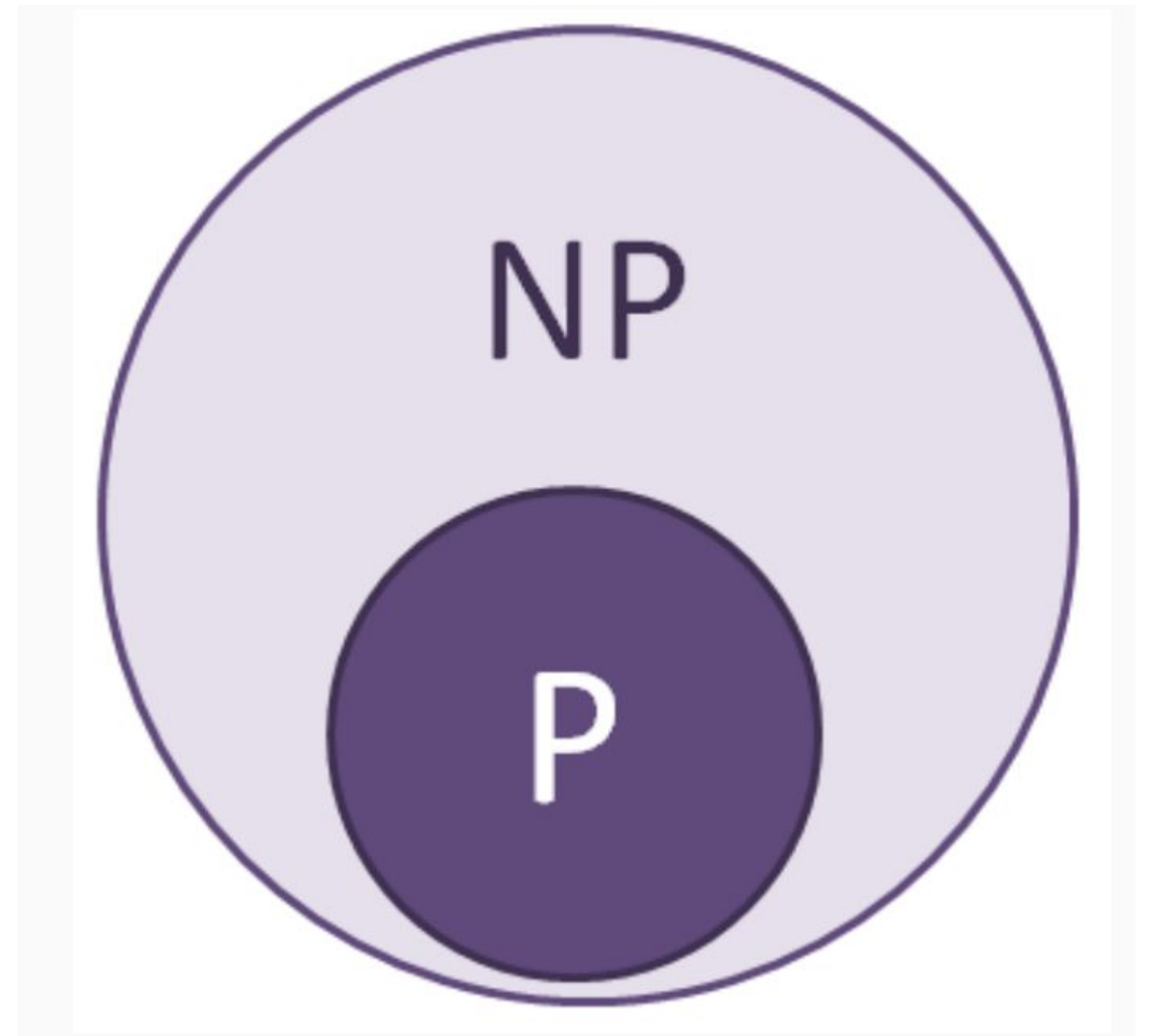
P vs NP



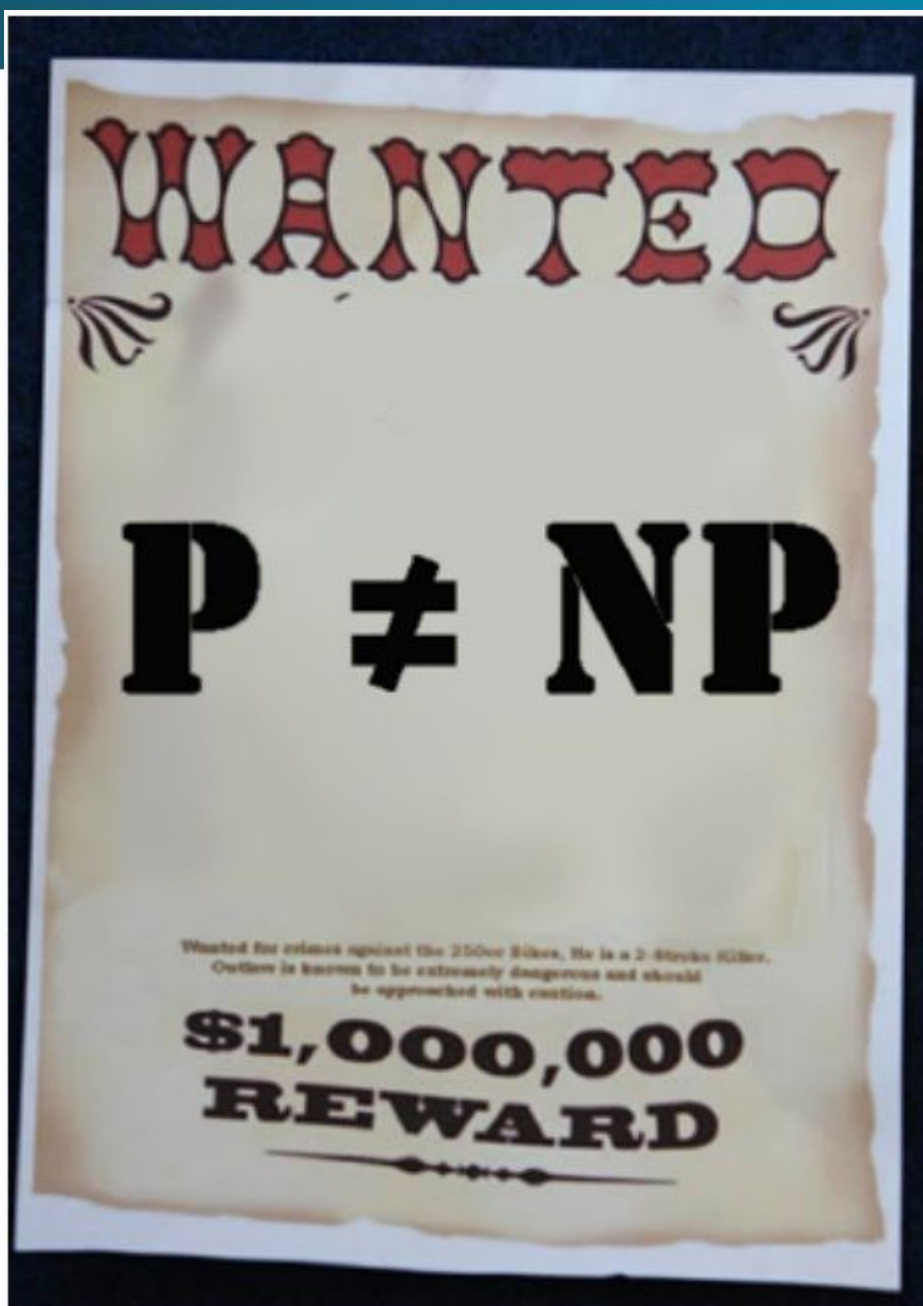
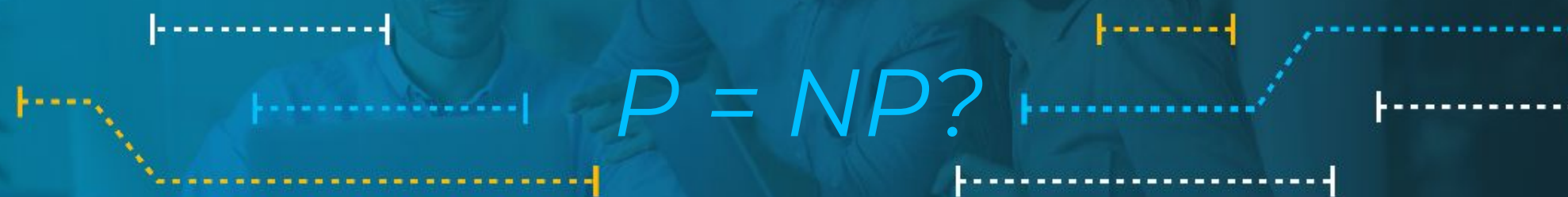
P $\subseteq$ **NP**, *trivial*

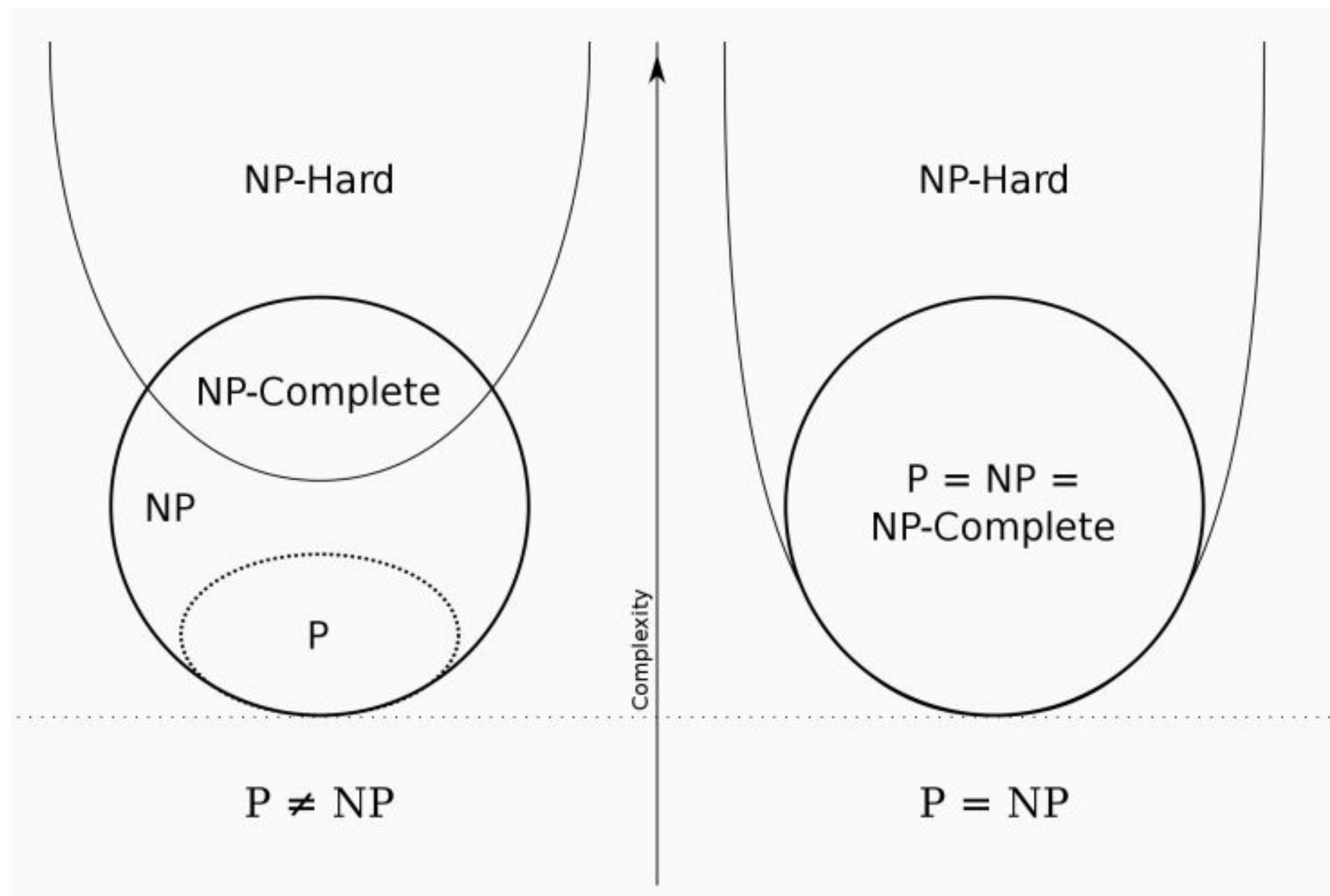
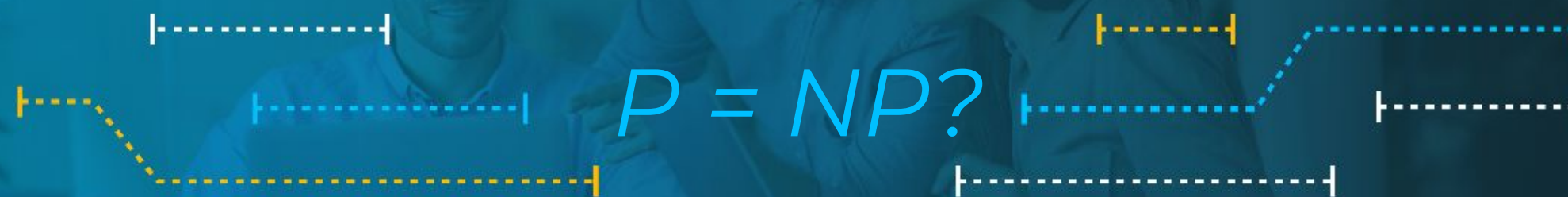


> Reinventa el mundo <



TRANSFORMATEC





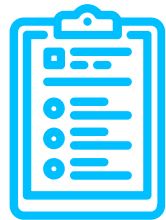
Computability *Theory*

- **Solvable vs unsolvable**
 - Solvable problems: All AED (and more)
 - Unsolvable problem:
 - Determine the veracity of a mathematical expression
 - Determine if a program stops after receive any input
- **The Halting problem**

Automata *Theory*

- **Study models of computation**
 - Finite Automaton
 - Pushdown Automaton
 - Context Free Grammar
 - Turing Machines
- They are all different models...
 - ***or are they?***

2.



Quick review

You should already know this

Mathematical Notations

- **Set**
 - **components:** elements
 - subset, proper subset, power set
 - **Types:**
 - multiset, infinite set, empty set, singleton set, unordered pair.
 - **Operations:**
 - Union, intersection, complement

Mathematical Notations

- **Sequence**
 - **components:** elements
 - subset, proper subset, power set
 - **Types:**
 - Finite(tuples):
 - k-tuples
 - ordered pair
 - Infinite
- **Sequence & Sets**
 - Cartesian product

Mathematical Notations

- **Function**(mapping)
 - Domain, Range
 - **Operations:**
 - Sum, rest, multiplication
 - Cartesian product

Mathematical Notations

- **Graph**
 - Undirected vs Directed
 - Node, edges
 - degree
 - subgraph

Mathematical Notations

- **Strings**

- **Alphabet(Σ):** Any nonempty finite set
- **Symbols:** elements of the alphabet
- **String over an Σ (ω):** Finite sequence of symbols from Σ
 - **Length ($|\omega|$):**
 - **Empty string: ϵ**
 - **Operations**
 - Reverse string ω^R
 - Concatenation
 - **Substring, Prefix,**

Mathematical Notations

- **Strings**
 - Σ^1 : Strings with size 1
 - Σ^n : Strings with size n
 - $\Sigma^+ : \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$
 - $\Sigma^* : \Sigma^0 \cup \Sigma^+$
 - **Language (L):**
 - A set of strings
 - Subset of Σ^*

Mathematical Proofs

- **Proofs**

- **Direct Proof:**

- $P \rightarrow Q$

- **Proof by induction:**

- $P(1)$, then $P(k) \rightarrow P(k+1)$ for all k

- **Proof by contradiction:**

- Assume P is false, then reach a contradiction

- **Pigeonhole Principle:**

- $+n$ pigeons in just n pigeonholes? At least 2 share the same pigeon

3.



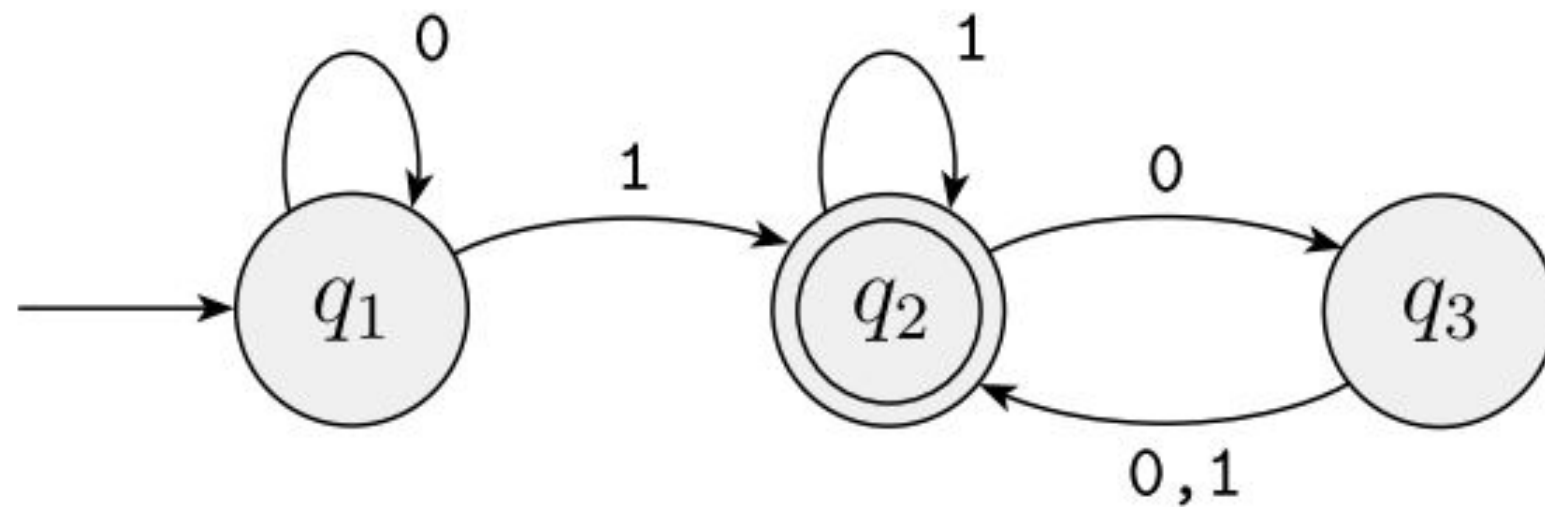
Automaton

Here we go again

Automata *Theory*

- **What is a computer?**
 - Complicated question
- We need to formalize the definition of computer
- First formalized an ideal version: **computational model**
 - **First model:**
 - Finite state machine, or **finite automaton**

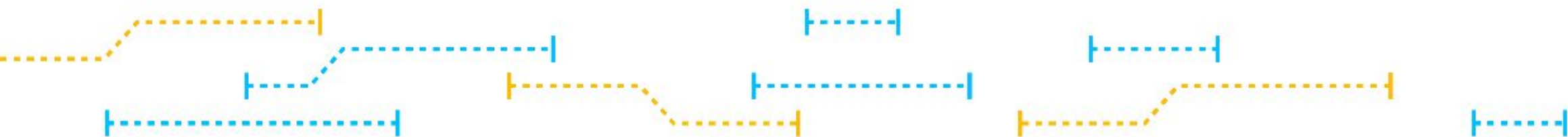
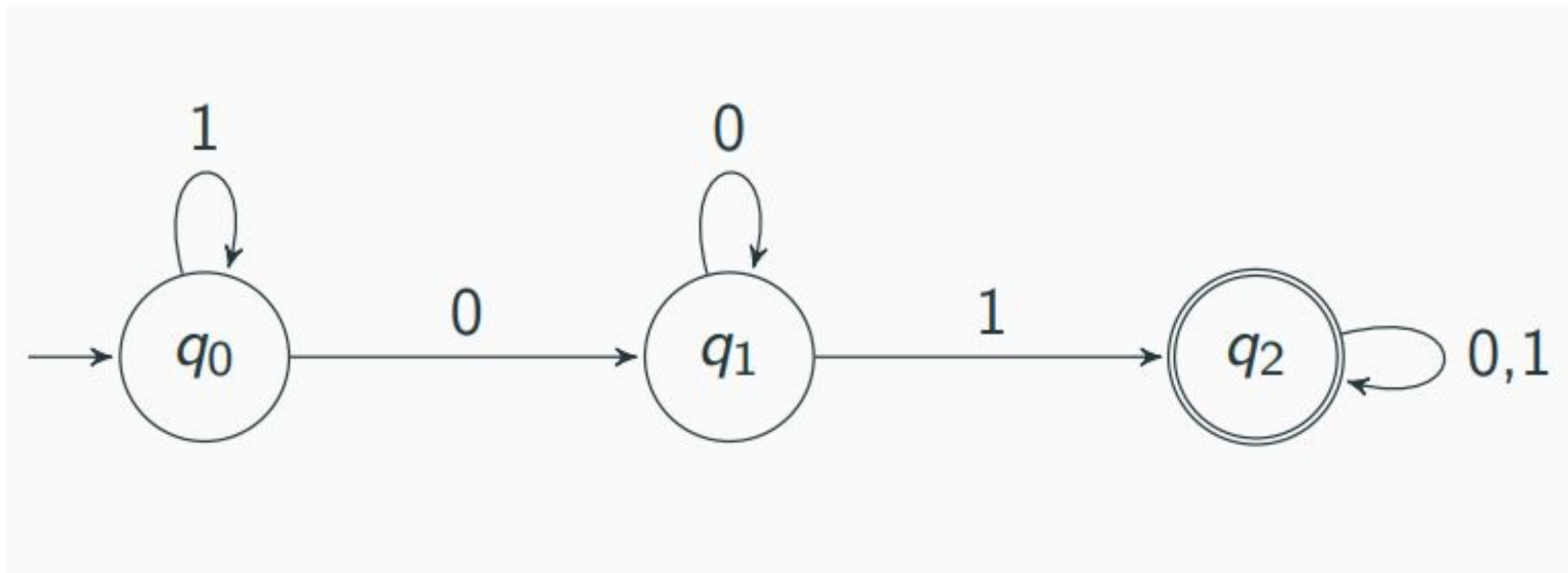
A (Deterministic Finite) *automaton* M_1



- **State Diagram:** Figure
- **States:** q_1, q_2, q_3
- **Start state:** q_1
- **Accept state:** q_2
- **Transitions:** the arrows



A (Deterministic Finite) *automaton* M_1



DFA

DEFINITION 1.5

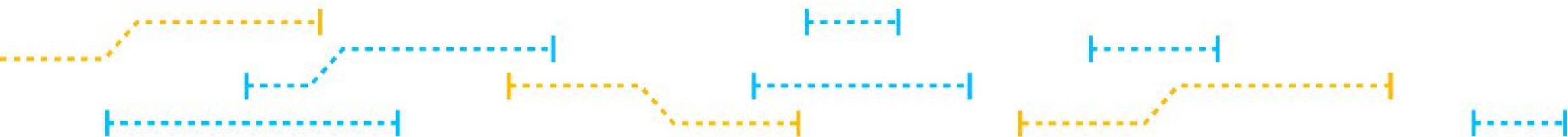
A *finite automaton* is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,¹
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.²

Formal Definitions

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton and let $w = w_1w_2 \cdots w_n$ be a string where each w_i is a member of the alphabet Σ . Then M ***accepts*** w if a sequence of states $r_0, r_1, \dots, r_n$ in Q exists with three conditions:

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$, for $i = 0, \dots, n - 1$, and
3. $r_n \in F$.



Automata *Theory*

- M **recognizes** a language L if
$$L = \{\omega \mid M \text{ accepts } \omega\}$$
- We also write **$L = L(M)$**
- A language L is called a **regular language** if there exists some finite Finite automaton M that recognizes it

Examples

- El lenguaje de todas las cadenas de 0's y 1's que consisten en n 0's seguidos de n 1's, para algún $n \geq 0$
 - **$\{\epsilon, 01, 0011, 000111, \dots\}$**
- El lenguaje de todas las cadenas de 0's y 1's que tienen la misma cantidad de 0's y 1's
 - **$\{\epsilon, 01, 10, 0011, 0101, 1001, \dots\}$**
- El lenguaje de todas las cadenas de 0's y 1's que representan números primos
 - **$\{10, 11, 101, 1011, \dots\}$**

Class Activity

Diseñe autómatas finitos para los siguientes lenguajes. El alfabeto es siempre $\{0, 1\}$

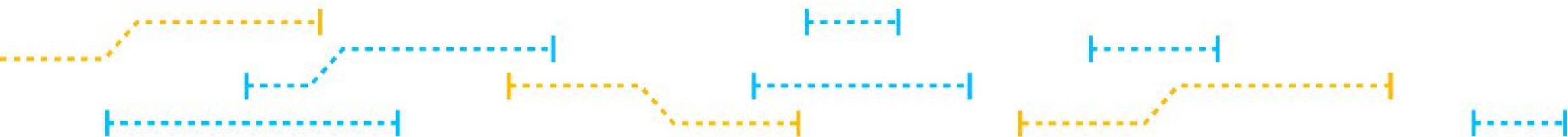
- a. Cadenas con un número impar de 1's
- b. Cadenas con un número par de 1's
- c. Cadenas con un número de unos múltiplo de 3
- d. Cadenas que tienen un número par de 0s siguiendo al último 1, y que tienen al menos un 1
- e. Cadenas que tienen al 001 como subcadena
- f. Cadenas que tienen al 011 como subcadena
- g. Cadenas con tres ceros consecutivos
- h. Cadenas cuyo segundo dígito empezando desde la derecha es 1



Class Activity

Diseñe autómatas finitos para los siguientes lenguajes. El alfabeto es siempre $\{0, 1\}$

- a. Cadenas que comienzan en 1 y terminan en 0
- b. Cadenas que tienen al 0101 como subcadena
- c. Todas las cadenas excepto la cadena vacía
- d. Cadenas cuyo tercer dígito empezando desde la derecha es 1
- e. $\{\epsilon, 0\}$
- f. El conjunto vacío
- g. Cadenas cuya longitud es máximo 5
- h. Cadenas con un número impar de 1's y un número impar de 0's



Regular Operations

- Let A and B two languages
- Regular operations
 - **Union:** $A \cup B = \{x : x \in A \text{ or } x \in B\}$
 - **Concatenation:** $A \cdot B = \{xy : x \in A, y \in B\}$
 - **Estrella:** $A^* = \{x_1x_2 \dots x_k : x_i \in A, \text{ and } k \geq 0\}$

Regular Operations

- $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$
 - $A \cup B = \{\text{good, bad, boy, girl}\}$
 - $A \cdot B = \{\text{goodboy, goodgirl, badboy, badgirl}\}$
 - $A^* = \{\epsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, ...}\}$

Regular Operations

- Let REGULAR, the set of regular languages L
 - Is REGULAR closed under the **union** operation?
 - Is REGULAR closed under **concatenation**?
 - Is REGULAR closed under **star** operation?

Theorem

REGULAR is closed under the union operation.

If L_1 and L_2 are two regular languages, then $L_1 \cup L_2$ is also a regular language

Proof Idea

- Proof by construction
- Since L_1 is regular, there exists a FDA A_1 that recognizes L_1
- Since L_2 is regular, there exists a FDA A_2 that recognizes L_2
- We will build A using A_1 and A_2

Proof Idea

- Let $L = L_1 \cup L_2$
- Let $A_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$
- Let $A_2 = \{Q_2, \Sigma, \delta_2, q_2, F_2\}$
- We define $A = \{Q, \Sigma, \delta, q, F\}$ as
- $Q = Q_1 \times Q_2$
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- $q_0 = (q_1, q_2)$
- $F = \{(r_1, r_2) : r_1 \in F_1 \text{ or } r_2 \in F_2\}$

Theorem

REGULAR is closed under concatenations.

If L_1 and L_2 are two regular languages, then $L_1 \cdot L_2$ is also a regular language

Proof Idea

- Spooky
- How can a FDA know the right partition to the string such that the first half is accepted by A1 and the second by A2?
- We need a more permissive model
- We need **NONDETERMINISM**

Thanks

Let's keep finding mistakes

